

Nova OS – Band 0: Nova Platform Specification

Version: 0.1

Status: Entwurf

Kurzname: NPS

1. Zweck

Band 0 definiert die technischen Grundregeln der gesamten Nova-Plattform.

Er gilt für:

- Bootloader
- Kernel
- Runtime
- Treiber
- Systemdienste
- Recovery
- SDK
- Anwendungen

2. Grundsatz

Alle Nova-Komponenten verwenden gemeinsame Datentypen, gemeinsame Fehlercodes, gemeinsame Versionierung und klar definierte Schnittstellen.

3. Basistypen

```
typedef unsigned char    nova_u8;
typedef unsigned short   nova_u16;
typedef unsigned int     nova_u32;
typedef unsigned long long nova_u64;

typedef signed char      nova_i8;
typedef signed short     nova_i16;
typedef signed int       nova_i32;
typedef signed long long nova_i64;

typedef nova_u64         nova_size;
typedef nova_u64         nova_address;
```

```
typedef nova_u64          nova_flags;  
typedef nova_u64          nova_handle;
```

4. Boolean

```
typedef enum  
{  
    NOVA_FALSE = 0,  
    NOVA_TRUE  = 1  
} nova_bool;
```

5. Ergebniswerte

Jede öffentliche Funktion gibt später bevorzugt einen `nova_status` zurück.

```
typedef enum  
{  
    NOVA_OK = 0,  
  
    NOVA_ERROR = 1,  
    NOVA_INVALID_ARGUMENT = 2,  
    NOVA_NOT_FOUND = 3,  
    NOVA_NO_MEMORY = 4,  
    NOVA_ACCESS_DENIED = 5,  
    NOVA_TIMEOUT = 6,  
    NOVA_NOT_SUPPORTED = 7,  
    NOVA_ALREADY_EXISTS = 8,  
    NOVA_BUSY = 9,  
    NOVA_CORRUPTED = 10,  
    NOVA_SECURITY_VIOLATION = 11  
  
} nova_status;
```

6. Versionierung

Alle stabilen Schnittstellen erhalten eine Version.

```
typedef struct  
{  
    nova_u16 major;  
    nova_u16 minor;  
    nova_u16 patch;
```

```
    nova_u16 build;
} nova_version;
```

Regel:

```
major ändert ABI-Kompatibilität
minor erweitert kompatibel
patch behebt Fehler
build beschreibt konkrete Builds
```

7. ABI-Strukturen

Jede ABI-Struktur beginnt mit einem gemeinsamen Header.

```
typedef struct
{
    nova_u32 magic;
    nova_u16 version_major;
    nova_u16 version_minor;
    nova_u32 struct_size;
    nova_u32 flags;
} nova_abi_header;
```

Damit kann der Kernel später prüfen:

- Ist die Struktur gültig?
- Welche Version liegt vor?
- Wie groß ist sie?
- Welche optionalen Funktionen sind aktiv?

8. Boot Context

Der Bootloader übergibt dem Kernel einen versionierten Boot Context.

```
typedef struct
{
    nova_abi_header header;

    nova_address framebuffer_address;
    nova_u32 framebuffer_width;
    nova_u32 framebuffer_height;
    nova_u32 framebuffer_pixels_per_line;
    nova_u32 framebuffer_bits_per_pixel;
```

```

nova_address memory_map_address;
nova_size memory_map_size;
nova_size memory_descriptor_size;

nova_u64 total_memory;
nova_u32 boot_flags;

} nova_boot_context;

```

9. Magic-Werte

Magic-Werte dienen zur schnellen Erkennung falscher Strukturen.

```

#define NOVA_MAGIC_BOOT_CONTEXT 0x4E424354
#define NOVA_MAGIC_MODULE      0x4E4D4F44
#define NOVA_MAGIC_DRIVER      0x4E445256

```

Bedeutung:

```

NBCT = Nova Boot Context
NMOD = Nova Module
NDRV = Nova Driver

```

10. Handle-Regel

Handles sind abstrakte Referenzen.

Ein Handle ist niemals direkt ein Speicherzeiger.

```

typedef nova_u64 nova_handle;

```

Ungültiger Handle:

```

#define NOVA_INVALID_HANDLE 0

```

11. Modulstatus

```

typedef enum
{

```

```
NOVA_MODULE_ENTWURF = 0,  
NOVA_MODULE_SPEZIFIZIERT = 1,  
NOVA_MODULE_IN_ENTWICKLUNG = 2,  
NOVA_MODULE_TESTBAR = 3,  
NOVA_MODULE_STABIL = 4,  
NOVA_MODULE_VERALTET = 5,  
NOVA_MODULE_ENTFERNT = 6  
} nova_module_status;
```

12. Speicheradressen

Nova OS unterscheidet klar:

```
physische Adresse  
virtuelle Adresse  
DMA-Adresse  
Framebuffer-Adresse  
Kernel-Adresse  
User-Adresse
```

Spätere Typen:

```
typedef nova_u64 nova_phys_addr;  
typedef nova_u64 nova_virt_addr;  
typedef nova_u64 nova_dma_addr;
```

13. Sichtbare Meldungen

Alle sichtbaren Meldungen sind auf Deutsch.

Beispiele:

```
[NOVA][INFO] Boot Context wurde erfolgreich geprüft.  
[NOVA][FEHLER] Ungültige ABI-Version.  
[NOVA][KRITISCH] Speicherkarte konnte nicht gelesen werden.
```

14. Kompatibilitätsregel

Eine neuere Komponente darf ältere Strukturen lesen, solange:

- `MAGIC` gültig ist

- `struct_size` mindestens die benötigten Felder enthält
 - `version_major` kompatibel ist
-

15. Erste offizielle Plattformdateien

Aus Band 0 entstehen als erste Dateien:

```
runtime/include/nova_types.h
runtime/include/nova_status.h
runtime/include/nova_version.h
runtime/include/nova_abi.h
runtime/include/nova_boot_context.h
runtime/include/nova_handle.h
```

16. Nächster Schritt

Als nächstes werden diese Header-Dateien vollständig ausgegeben.

Danach bauen Bootloader und Kernel ausschließlich auf diesen gemeinsamen Plattfortmtypen auf.